

How to Run Re-DiffiNet for Breast Tumor Segmentation

Here's a step-by-step guide on how to set up and run the Re-DiffiNet code, including where to place your input images.

Prerequisites

First, make sure you have the following dependencies installed:

```
pip install torch torchvision matplotlib numpy scipy pillow monai tqdm
```

Directory Structure

Before running the code, ensure your project has the following structure:

```
ReDiffinet-BreastSeg/
├─ config/
│   └─ config.py
├─ data/
│   ├── dataset.py
│   └─ transforms.py
├─ models/
│   ├── unet.py
│   ├── diffusion.py
│   └─ reddenet.py
├─ utils/
│   ├── metrics.py
│   └─ visualization.py
├─ train.py
└─ inference.py

# Data directory (will need to be created)
├─ dataset/
│   ├── train/
│   │   ├── images/
│   │   └─ masks/
│   ├── val/
│   │   ├── images/
│   │   └─ masks/
│   └─ test/
```

```
|      |— images/
|      |— masks/
```

```
# These directories will be created automatically during execution
```

```
|— weights/
|— visualizations/
|— results/
```

Setting Up Input Images

For Training

1. Place your breast images (mammograms or MRIs) in the following directories:
 - Training images: `dataset/train/images/`
 - Training masks: `dataset/train/masks/` (segmentation ground truth)
 - Validation images: `dataset/val/images/`
 - Validation masks: `dataset/val/masks/`
2. Make sure each image and its corresponding mask have the same filename.

For Inference

For inference on new images, you have two options:

1. Place test images in `dataset/test/images/`
2. Specify individual image paths in the `inference.py` script

Configuring the Code

Update the Configuration

Open `config/config.py` and modify the settings:

```
class Config:
    # Update this to point to your dataset directory
    data_path = "./dataset" # Change this to your data directory path

    # Adjust these parameters based on your images
    image_size = (256, 256) # Change to your desired image size
    unet_in_channels = 1    # 1 for grayscale, 3 for RGB

    # Other parameters can be adjusted as needed
    batch_size = 4
    learning_rate = 1e-4
    num_epochs = 100
```

```
# Device - change to 'cpu' if you don't have a CUDA GPU
device = "cuda" if torch.cuda.is_available() else "cpu"
```

Update Inference Script

Open `inference.py` and modify line 114 to point to your test image:

```
# Example: Run inference on a single image
image_path = "./dataset/test/images/your_test_image.png" # Replace with
actual path
```

Running the Code

Training

1. First, train the model:

```
python train.py
```

This will:

- Train the baseline U-Net
- Train the Re-DiffiNet model
- Save model weights to the `weights/` directory
- Save visualization samples to the `visualizations/` directory

Inference

2. To run inference on test images:

```
python inference.py
```

This will:

- Load the trained model
- Process the specified test image
- Save results to the `results/` directory

Batch Inference

If you want to run inference on multiple images, you can modify the `inference.py` script to loop through all images in your test directory:

```

def main():
    # Previous code...

    # Create results directory
    results_dir = os.path.join(os.path.dirname(__file__), 'results')
    os.makedirs(results_dir, exist_ok=True)

    # Run inference on all images in test directory
    test_dir = os.path.join(config.data_path, 'test', 'images')
    for image_file in os.listdir(test_dir):
        if image_file.endswith(('.png', '.jpg', '.tif')):
            image_path = os.path.join(test_dir, image_file)
            print(f"Processing {image_path}...")

            results = run_inference(model, image_path, device)

            # Save results
            image_name = os.path.splitext(os.path.basename(image_path))[0]

            # Save U-Net prediction
            unet_save_path = os.path.join(results_dir, f"
{image_name}_unet.png")
            save_segmentation_overlay(
                results["image"],
                results["unet_pred"],
                unet_save_path
            )

            # Save refined prediction
            refined_save_path = os.path.join(results_dir, f"
{image_name}_refined.png")
            save_segmentation_overlay(
                results["image"],
                results["refined_pred"],
                refined_save_path
            )

            print(f"Results saved to {results_dir}")

```

Visualizing Results

After running inference, check the `results/` directory for:

- `[image_name]_unet.png` : Segmentation from the baseline U-Net
- `[image_name]_refined.png` : Refined segmentation from Re-DiffiNet

The segmentation overlays will show:

- Red: First class (e.g., mass)
- Green: Second class (e.g., calcification)
- Blue: Third class (e.g., architectural distortion)

Troubleshooting

- If you encounter memory issues, reduce the `batch_size` in `config.py`
- If using CPU instead of GPU, you may need to reduce the diffusion steps by setting `diffusion_steps` to a lower value (e.g., 100) in `config.py`